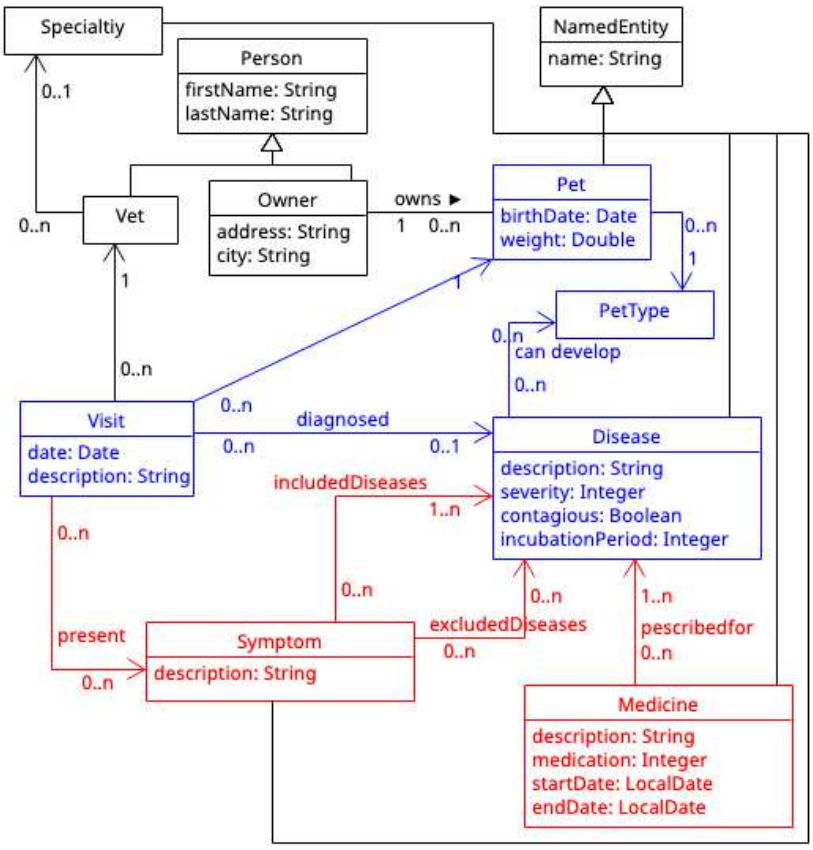
Control práctico de DP1 2024-2025 (Control-check 2)

Enunciado

En este ejercicio, añadiremos la funcionalidad de gestión de enfermedades, síntomas y medicamentos. Concretamente, se proporciona una clase “Disease” que representa a las enfermedades que pueden desarrollar las mascotas, que se relaciona con el tipo de mascotas que pueden sufrirlas. Esta clase, además de la descripción y severidad de la enfermedad, presenta los atributos de contagiosidad y de periodo de incubación de la dolencia. Además, tendremos las clases “Symptom” y “Medicine” que representan a los síntomas que pueden aparecer a consecuencia de una enfermedad y los medicamentos recomendados para cada enfermedad respectivamente. Además, se ha creado una relación que indica qué síntomas son susceptibles de presentarse para una enfermedad llamada “includedDiseases”, y otra relación para indicar qué síntomas excluyen que se trate de ciertas enfermedades llamada “excludedDiseases”, para ayudar a los veterinarios a realizar diagnósticos más precisos. Por otro lado, tenemos una relación denominada “prescribedfor” para indicar la medicación recetada para la cura de la afección de la mascota.El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:



Las clases para las que realizaremos el mapeo como entidades JPA se han señalado en rojo. Las clases azules son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control.

Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias. Si desea ver el resultado que arrojarían las pruebas en backend, puede ejecutarlas (bien mediante su entorno de desarrollo favorito, bien mediante el comando “*mvnw*

test" en la carpeta raíz del proyecto). Cada ejercicio correctamente resuelto valdrá dos puntos, el número de casos de prueba de cada ejercicio puede variar entre uno y otro y la nota se calculará en base al porcentaje de casos de prueba que pasan. Por ejemplo, si pasan la mitad (50%) de los casos de prueba de un ejercicio, en lugar de dos puntos usted obtendrá un 1.

Para comenzar el control debe aceptar la tarea de este control práctico a través del siguiente enlace:

https://classroom.github.com/a/c6rYpQqz

Al aceptar dicha tarea, se creará un repositorio único individual para usted, debe usar dicho repositorio para realizar el control práctico. Debe entregar la actividad en EV asociada al control check proporcionando como texto la dirección url de su repositorio personal. Recuerde que además debe entregar su solución del control.

La entrega de su solución al control se realizará mediante un único comando "*git push*" a su repositorio individual. Recuerde que debe hacer push antes de cerrar sesión en la computadora y abandonar el aula, de lo contrario, su intento se evaluará como no presentado. Su primera tarea en este control será clonar (recuerde que si va a usar los equipos del aula para realizar el control necesitará usar un token de autenticación de GitHub como clave, tiene un documento de ayuda a la configuración en el propio repositorio del control). A continuación, deberá importar el proyecto en su entorno de desarrollo favorito y comenzar los ejercicios abajo listados. Al importar el proyecto, el mismo puede presentar errores de compilación. No se preocupe, si existen, dichos errores irán despareciendo conforme usted vaya implementando los distintos ejercicios del control.

Nota importante 1: No modifique los nombres de las clases ni la signatura (nombre, tipo de respuesta y parámetros) de los métodos proporcionados como material de base para el control. Las pruebas que se usan para la evaluación dependen de que las clases y los métodos tengan la estructura y nombres proporcionados. Si los modifica probablemente no pueda hacer que pasen las pruebas.

Nota importante 2: No modifique las pruebas unitarias proporcionadas como parte del proyecto bajo ningún concepto. Aunque modifique las pruebas en su copia local del proyecto, éstas serán restituidas mediante un comando git previamente a la ejecución de las pruebas para la emisión de la nota final, por lo que sus modificaciones en las pruebas no serán tenidas en cuenta en ningún momento.

Nota importante 3: Mientras haya ejercicios no resueltos habrá tests que no funcionen y, por tanto, el comando "*mvnw install*" finalizará con error. Esto es normal debido a la forma en la que está planteado el control y no hay que preocuparse por ello. Si se quiere probar la aplicación se puede ejecutar de la forma habitual pese a que "mvnw install" finalice con error.

Nota importante 4: La descarga del material de la prueba usando git, y la entrega de su solución con git a través del repositorio GitHub creado a tal efecto forman parte de las competencias evaluadas durante el examen, por lo que no se aceptarán entregas que no hagan uso de este medio, y no se podrá solicitar ayuda a los profesores para realizar estas tareas.

Nota importante 5: No se aceptarán como soluciones válidas proyectos cuyo código fuente no compile correctamente o que provoquen fallos al arrancar la aplicación en la inicialización del contexto de Spring. Las soluciones cuyo código fuente no compile o sean incapaces de arrancar el contexto de Spring serán evaluadas con una nota de 0.

Test 1 – Creación de las entidades Symptom y Medicine y sus repositorios asociados

Parte 1A: Entidades Symptom y Medicine y sus repositorios (1punto):

Modificar las clases “Symptom” y “Medicine” para que sean entidades. Estas clases están alojadas en el paquete “org.springframework.samples.petclinic.disease”, y deben tener los siguientes atributos y restricciones:

Para ambas clases:

- El atributo de tipo entero (Integer) llamado “id” actuará como clave primaria en la tabla de la base de datos relacional asociada a la entidad.
- Un atributo de tipo cadena de caracteres (String) llamado “name” obligatorio (no puede ser nulo), que debe tener una longitud mínima de 3 caracteres y máxima de 50 y que no puede estar formada por caracteres vacíos (espacios, tabuladores, etc.).
- El atributo de tipo cadena caracteres (String) llamado “description” opcional.

Para la clase Medicine:

- El atributo de tipo entero (Integer) llamado “medication”, que representa la cantidad de medicación que debe tomar el animal en función de su peso. El atributo será obligatorio y tendrá un valor mínimo de 1 y un valor máximo de 1000 (a partir de este valor, la dosis puede ser mortal).
- El atributo de tipo fecha (LocalDate) llamado “startDate”, que representa la fecha en que comienza la medicación pautada. La fecha seguirá el formato “dd/MM/yyyy” (puede usar como ejemplo la clase Pet y su fecha de nacimiento para ver cómo se especificar dicho formato, pero nótese que el patrón del formato es distinto). Este atributo es obligatorio.
- El atributo de tipo fecha (LocalDate) llamado “endDate”, que representa la fecha en que debe terminar la medicación pautada. Seguirá el formato “dd/MM/yyyy”. Este atributo debe ser obligatorio.

No modifique por ahora las anotaciones @Transient de las clases. Modificar las interfaces “SymptomRepository” y “MedicineRepository” alojadas en el mismo paquete para que extiendan a CrudRepository. No olvide especificar sus parámetros de tipo.

Parte 1B: Relaciones entre las entidades (1 punto)

Elimine las anotaciones @Transient de los métodos y atributos que las tengan en las entidades creadas en el ejercicio anterior, así como la del atributo symptoms de la clase Visit. Se pide crear las siguientes relaciones entre las entidades:

Cree una relación unidireccional desde “Visit” hacia “Symptom” que exprese la que aparece en el diagrama UML (mostrado en la primera página de este enunciado) respetando sus cardinalidades, usando el atributo “symptoms” de la clase “Visit”.

Además, se pide crear dos relaciones unidireccionales desde “Symptom” hacia “Disease” que representen las que aparecen en el diagrama UML, tenga en cuenta la cardinalidad que tienen (*recuerde que en este caso, al tratarse de una doble relación n a n entre las mismas entidades se trata de unas relaciones bastante exóticas)*, usando como nombre de los atributos “includedDiseases” y “excludedDiseases” en la clase “Symptom”.

Finalmente, se pide crear una relación unidireccional desde “Medicine” hacia “Disease” que represente la que aparece en el diagrama. Debe asegurarse de que las relaciones expresan adecuadamente la cardinalidad que muestra el diagrama UML.

Test 2 – Modificación del script de inicialización de la base de datos

Parte 2A: Incluir dos síntomas y dos medicaciones (1 punto)

Modificar el script de inicialización de la base de datos, para que se creen los siguientes síntomas (Symptom) y medicaciones (Medicine):

Symptom 1:

- id: 1
- name: “Cough”
- description: null

Symptom 2:

- id: 2
- name: “Hair loss”
- description: “Hair loss in animals, also known as alopecia, can be a common and concerning symptom with various potential underlying causes.”

Medicine 1:

- id: 1
- name: “aspirin”
- description: “Aspirin, also known by its generic name acetylsalicylic acid, is a widely used medication with analgesic (pain-relieving), antipyretic (fever-reducing), and anti-inflammatory properties.”
- medication: 500
- startDate: 17/12/2024
- endDate: 25/12/2024

Medicine 2:

- id: 2
- name: “paracetamol”
- description: “Paracetamol, known as acetaminophen in the United States and Canada, is a widely used over-the-counter (OTC) medication with analgesic (pain-relieving) and antipyretic (fever-reducing) properties.”
- medication: 800
- startDate: 3/12/2024
- endDate: 15/12/2024

Tenga en cuenta que el orden en que aparecen los INSERT en el script de inicialización de la base de datos es relevante al definir las asociaciones. El formato para introducir fechas en SQL que debe usar es similar al que expresa esta cadena:'2020-12-30'.

Parte 2B: Relacionar las medicaciones y los síntomas con las visitas y enfermedades (1 punto)

Modificar este script de inicialización de la base de datos para que:

- La Medicine cuyo id es 1 (Aspirin) se asocie con la Disease cuyo id es 3 (Flu).
- La Medicine cuyo id es 2 (Parecetamol) se asocie con la Disease cuyo id es 1 (Rabies).
- El Symptom cuyo id es 1 tenga como posibles enfermedades incluidas la Diseases con ids 3 y 5.
- El Symptom cuyo id es 2 tenga como posibles enfermedades incluidas la Diseases con ids 4 y 5.
- El Symptom cuyo id es 1 excluya como posibles enfermedades las Diseases con id 1 y 2.
- El Symptom cuyo id es 2 excluya como posible enfermedad la Disease con id 3.
- La Visit cuyo id es 1 tenga asociados los síntomas 1 y 2.

Tenga en cuenta que el orden en que aparecen los INSERT en el script de inicialización de la base de datos es relevante al definir las asociaciones

Test 3 – Creación y modificación de un controlador y un componente frontend de visualización de la historia clínica de las mascotas

Parte 3A – Creación de controlador de historia clínica de las mascotas (1 punto)

Modificar la clase "HistoryController" para que responda a peticiones tipo GET en la url: http://localhost:8080/api/v1/pets/{petID}/history

Para ello, el controlador debe usar el servicio de gestión de visitas (VisitService).

Es importante que dicho controlador devuelva los datos de las visitas y las enfermedades asociadas a la mascota en el siguiente formato:

```
{
        petName: "Leo",
        visits: [   {visitDate: "12/10/2024", disease: null},
                    …,
                    {visitDate: "18/11/2024", disease: "Flu"}    ]
}
```

Este endpoint de la API debería estar accesibles únicamente para usuarios de tipo Vet o de tipo Owner (que tengan la authority "VET" o bien la authority "OWNER"). Si se pide la historia de un pet cuyo id no está en la BD se deberá devolver un código de respuesta 404.

Parte 3B - Creación de un componente frontend para la visualización de la historia clínica de las mascotas (1 punto)

Modificar el componente React proporcionado en el fichero "frontend/src/disease/history/index.js" para que muestre un listado de las visitas y enfermedades diagnosticadas para una mascota. Para ello debe hacer uso de la API lanzando una petición tipo GET contra la URL api/v1/pets/{petID}/history.

Este componente debe tomar como propiedad llamada id el identificador de la mascota para la que se debe mostrar la historia. Tras realizar la llamada a la API, el componente únicamenet debe mostrar el nombre de la mascota como título de nivel 1 (h1) y una tabla con las fechas de las visitas (en una columna con título “Date”) y los nombres de las enfermedades diagnosticadas (en una columna titulada “Disease”), se recomienda usar el componente Table de reactstrap. No incluya etiquetas adicionales como listas, divs, spans, etc.

Para poder lanzar esta prueba y comprobar su resultado puede colocarse en la carpeta de frontend y ejecutar el comando npm test y pulsar 'a’ en el menú de comandos de jest. Nótese que previamente debe haber lanzado al menos una vez el comando npm install para que todas las librerías de node estén instaladas

Test 4 – Anotar el repositorio de Enfermedades con una consulta compleja

Modificar la consulta personalizada que puede invocarse a través del método “findEpidemicDiseases” del repositorio de tratamientos “DiseaseRepository” (alojado en el paquete org.springframework.samples.petclinic.disease) que reciba como parámetro un conjunto de tipos de mascotas, dos fechas determinadas (que definen un rango de fechas donde la primera es anterior a la segunda), y un entero que representa un número de diagnósticos. El objetivo es que devuelva todas las enfermedades diagnosticadas entre los tipos de mascotas especificados en alguna visita durante el periodo especificado, tales que estas enfermedades han sido diagnosticadas como mínimo el número de veces que indica el último parámetro. Este método permitirá encontrar enfermedades con visos de epidemia entre la población de una serie de tipos de mascotas durante un periodo indicado.

A continuación, se muestra un ejemplo, sea la siguiente tabla de visitas y diagnósticos entre el 12 de noviembre de 2024 y el 17 de diciembre de 2024:

Pet	PetType[1]	Date (from Visit)	Diagnose
{id:1, name:“Leo”}	{id: 1, name:”Cat”}	13/11/2024	{id:4,name:”Toxoplasmosis”}
{id: 3, name: “Rosy”}	{id: 2, name:”Dog”}	14/11/2024	{id:1,name:”Rabies Shot”}
{id: 2, name:”Basil”}	{id: 6, name:”Hamster”}	17/11/2024	{id:1,name:”Rabies Shot”}
{id: 4, name:”Jewel”}	{id: 2, name:”Dog”}	18/11/2024	{id:1,name:”Rabies Shot”}
{id:10,name:”Mulligan”}	{id: 2, name:”Dog”}	29/11/2024	{id:5,name:”Lyme Disease”}
{id:12, name:”Lucky”}	{id: 2, name:”Dog”}	05/12/2024	{id:3,name:”Flu”}
{id:12, name:”Lucky”}	{id: 2, name:”Dog”}	10/12/2024	{id:1,name:”Rabies Shot”}
{id:8, name:”Max”}	{id: 1, name:”Cat”}	16/12/2024	{id:1,name:”Rabies Shot”}

Si invocamos al método con los siguientes valores de los parámetros: petTypes={“dog”,”cat”}, startDate=12 de noviembre de 2024, endDate=16 de diciembre de 2024, diagnoses=2. El resultado debería ser un conjunto que contenga la enfermedad llamada “Rabies Shot” puesto que para perros y gatos en esas fechas hubo siete visitas con diagnóstico, y cuatro de ellas fueron de dicha enfermedad. Nótese que las enfermedades llamadas “Flu”, “Toxoplasmosis” y “Lyme Disease” no aparecerían puesto que durante el periodo indicado solamente se diagnosticaron una vez para el tipo de mascotas analizados.

[1] Obtained for the pet of the corresponding visit

Test 5 – Implementar una prueba para un algoritmo de validación de la aplicación de la medicación

En la clínica se ha decidido crear un sistema informático que permita validar la medicación a aplicar a las mascotas por el personal antes de suministrarlo. Para ello el algoritmo tomará como parámetros un valor de dosis de medicación máxima a aplicar, una mascota, una enfermedad, y la medicina concreta que se aplicará (que incluirá la cantidad de medicación y fecha de fin del tratamiento). El algoritmo debe comprobar que, para dosis de medicación superiores a lo indicado por el primer parámetro del método (por ejemplo, 600), para perros con enfermedades contagiosas, a día del suministro de la dosis inicial (usar el día actual como día de comienzo de la toma de la primera dosis) al animal no se le puede pautar más de 7 días de medicación (tomar como día de finalización del tratamiento la fecha de fin del tratamiento especificada en la medicina). En caso contrario, se lanzará una excepción de tipo IncorrectDoseException.

La interfaz del algoritmo está especificada en la interfaz "MedicineValidatorAlgorithm" que se encuentra en el paquete "org.springframework.samples.petclinic.disease.medicinevalidator". Se proporcionan cinco implementaciones del algoritmo (una correcta y cuatro incorrectas).

Modifique la clase de pruebas llamada "MedicineValidatorTest" que se encuentra en la carpeta "scr/main/test/disease" y especifique tantos métodos con casos de prueba como considere necesarios para validar el correcto funcionamiento del algoritmo. Nótese que la clase tiene un atributo de tipo MedicineValidatorAlgorithm llamado algorithm, use dicho atributo como sujeto bajo prueba en todos sus métodos de prueba.

Su implementación del test **no debe usar mocks, ni anotaciones de pruebas de spring** (@DataJpaTest, @SpringBootTest,etc.), **ni tests parametrizados, y todos los métodos anotados con @Test deben ser sin parámetros**.

CONTROL CHECK 2 - GRUPO 1 - 24/25

TEST 1

```java
@Getter
@Setter
@Entity
public class Symptom extends BaseEntity {

    @NotNull
    @NotBlank
    @Size(min = 3, max = 50)
    String name;

    String description;

    @ManyToMany
    @NotNull
    Set<Disease> includedDiseases;

    @ManyToMany
    Set<Disease> excludedDiseases;

}
```

```java
@Getter
@Setter
@Entity
public class Medicine extends BaseEntity {

    @NotNull
    @NotBlank
    @Size(min = 3, max = 50)
    String name;

    String description;

    @NotNull
    @Min(1)
    @Max(1000)
    Integer medication;

    @NotNull
    @DateTimeFormat(pattern = "dd/MM/yyy")
    LocalDate startDate;
```

```
    @NotNull
    @DateTimeFormat(pattern = "dd/MM/yyy")
    LocalDate endDate;

    @ManyToMany
    @NotNull
    Set<Disease> prescribedfor;

}
```

```
public interface SymptomRepository extends
CrudRepository<Symptom, Integer> {

    Optional<Symptom> findById(Integer i);

    List<Symptom> findAll();

    Symptom save(Symptom any);

}
```

```
public interface MedicineRepository extends
CrudRepository<Medicine, Integer> {

    Optional<Medicine> findById(Integer i);

    List<Medicine> findAll();

    Medicine save(Medicine any);

}
```

TEST 2

```
INSERT INTO medicine_prescribedfor(medicine_id,
prescribedfor_id) VALUES (1, 3), (2, 1);
INSERT INTO symptom_included_diseases(symptom_id,
included_diseases_id) VALUES (1, 3), (1, 5), (2, 4), (2, 5);
INSERT INTO symptom_excluded_diseases(symptom_id,
excluded_diseases_id) VALUES (1, 1), (1, 2), (2, 3);
INSERT INTO visits_symptoms(visit_id, symptoms_id) VALUES (1,
1) ,(1, 2);
```

TEST 3

```
@RestController
public class HistoryController {

   PetService petService;
   VisitService visitService;

   @Autowired
   public HistoryController(PetService petService,
VisitService visitService){
       this.petService = petService;
       this.visitService = visitService;
   }
   @GetMapping("/api/v1/pets/{petId}/history")
   @ResponseStatus(HttpStatus.OK)
   public ResponseEntity<HistoryDTO>
getPetHistory(@PathVariable Integer petId){
       Pet pet = petService.findPetById(petId);
       if (pet == null ) throw new
ResourceNotFoundException("pet not found");

       Collection<Visit> visits =
visitService.findVisitsByPetId(petId);

       HistoryDTO res = new HistoryDTO(pet.getName(), visits);
       return new ResponseEntity<>(res, HttpStatus.OK);


   }
```

```
@Getter
@Setter
public class HistoryDTO {
   String petName;
   List<VisitDTO> visits;

   public HistoryDTO(String petName, Collection<Visit>
visits){
       this.petName = petName;
       this.visits = visits.stream().map(v -> new
VisitDTO(v)).toList();
   }
}
```

```
@Getter
@Setter
public class VisitDTO {
   String disease;
   LocalDate visitDate;

   public VisitDTO(Visit visit){
       this.disease = visit.getDiagnose().getName();
       this.visitDate = visit.getDatetime().toLocalDate();
   }

}
```

SecurityConfiguration.java

```
.requestMatchers(AntPathRequestMatcher.antMatcher("/api/v1/pets/**")).hasAnyAuthority("OWNER", "VET")
```

TEST 4

```
@Query("""
       SELECT d
       FROM Disease d
       JOIN Visit v ON v.diagnose = d
         WHERE v.pet.type IN :petTypes
         AND v.datetime BETWEEN :startDate AND :endDate
       GROUP BY d.id\s
       HAVING COUNT(v.id) >= :diagnoses

       """
)
Set<Disease> findEpidemicDiseases(@Param("petTypes")
Set<PetType> petTypes,
                                  @Param("startDate")
LocalDateTime startDate,
                                  @Param("endDate")
LocalDateTime endDate,
                                  @Param("diagnoses")Integer
diagnoses);
```

TEST 5

```
@Test
public void someTest(){
   // Arrangement / Configuration /Fixture
    Medicine m1=createMedicine("Ibuprofen","For
pain",800,LocalDate.of(2025, 1, 28));
    Pet p=createValidPet(DOG);
    Pet p2 = createValidPet(CAT);

   Medicine m2 =createMedicine("Ibuprofen","For
pain",400,LocalDate.of(2025, 1, 22));

   Assertions.assertThrows(IncorrectDoseException.class, () ->
algorithm.validateDose(600, p, disease1, m1));
   Assertions.assertDoesNotThrow(()->
algorithm.validateDose(600, p, disease1, m2));

   Assertions.assertThrows(IncorrectDoseException.class, () ->
algorithm.validateDose(600, p, disease4, m1));
   Assertions.assertDoesNotThrow(()->
algorithm.validateDose(600, p, disease4, m2));

   Assertions.assertDoesNotThrow(()->
algorithm.validateDose(600, p, disease2, m1));
   Assertions.assertDoesNotThrow(()->
algorithm.validateDose(600, p, disease3, m1));

   Assertions.assertDoesNotThrow(()->
algorithm.validateDose(600, p2, disease1, m2));
   Assertions.assertDoesNotThrow(()->
algorithm.validateDose(600, p2, disease1, m1));
}
```